MORTIS: COMPILER-GUIDED KV-CACHE MANAGEMENT FOR SERVING LLM AGENT PROGRAMS

Anonymous Authors[1]

ABSTRACT

Existing agent serving systems divide each prompt into a reusable prefix and an opaque suffix, managing the KV-cache at this coarse granularity without distinguishing the components within each part. We observe that prompt components differ in which LLM calls share their values and how long those values remain reusable, a property we term *intra-prompt heterogeneity*. Our key insight is that, in frameworks that generate prompts from typed LLM function calls, these components correspond to program variables, allowing compiler analysis to recover their sharing patterns and lifetimes. We present MORTIS, a compiler-serving co-design that combines this analysis with runtime observations to improve KV-cache management. MORTIS reorders prompt components to expose additional prefix reuse and uses their predicted reuse and lifetimes to guide cache retention, eviction, and prefetching. Our evaluation shows that MORTIS achieves up to 1.99× job completion time (JCT) speedup and 1.97× throughput improvement over SGLang, with geometric-mean speedups of 1.45× and 1.49×, respectively.

1 INTRODUCTION

Agent applications solve tasks through sequences of dependent LLM calls that repeatedly consume common context, including instructions, input documents, and intermediate results. Yet repeated content does not automatically translate into reusable computation. Processing a prompt, known as *prefill*, produces attention states stored in a key-value (KV) cache. A serving system can reuse these states when a later prompt begins with the same sequence of tokens, called a *shared prefix*. The remaining portion of the prompt is its *suffix*.

Whether reuse improves performance also depends on cache management. GPU memory is limited, and retaining cached states for one call can displace states needed by another. Evicting a prefix before its next use forces recomputation, while retaining context that will not be reused wastes scarce memory. Under KV-cache thrashing, per-request prefill time can grow by 6.8× and generation throughput can drop by up to 66% (Yuan et al., 2026). Efficient serving therefore requires both exposing reusable prefixes and deciding which cached states deserve memory between calls.

Existing systems exploit prefix sharing and workflow predictability. SGLang's RadixAttention reuses the longest cached prefix matching each incoming prompt and prioritizes requests with longer matches (Zheng et al., 2024). Workflow-aware systems such as KVFlow, Helium, PBKV, and CacheScout (Pan et al., 2025; Wadlom et al., 2026; Zheng et al., 2026; Zhang et al., 2026) use workflow structure or observed execution patterns to anticipate upcoming calls and guide cache management. KVFlow, for example, uses predicted agent execution order to prioritize retention of fixed prompt prefixes, while assigning varying suffixes the highest eviction priority. This policy treats the suffix as opaque with respect to its components' reuse potential: it does not distinguish persistent shared values from temporary content within that suffix.

However, we observe that the dynamic suffix has internal structure: it comprises distinct components with different sharing patterns and lifetimes, a property we term intra-prompt heterogeneity. A session-wide input, an accumulating history, and a temporary result may appear together in one prompt, yet have very different reuse potential. Some values recur across several calls, some preserve earlier content as they grow, and others cease to be needed after a single iteration. Recognizing this structure creates opportunities to expose sharing hidden by the current prompt order and to distinguish useful cached states from those that can be released.

Consider the fact-checking agent in Figure 1. Given a claim, the agent generates a query (A), searches for evidence (B), and assesses the result against previously collected evidence (C). It repeats the search and assessment until it can produce a verdict (D). In this example, prior systems reuse fixed instruction prefixes consisting of the common system

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

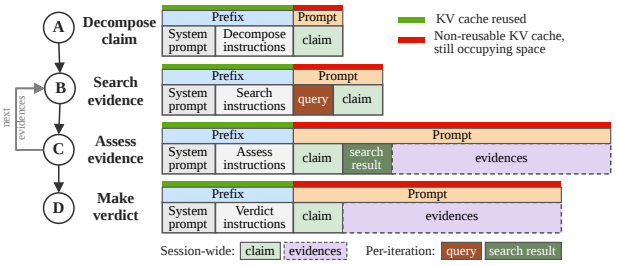


Figure 1. Original prompt layouts for a fact-checking agent. Call-specific instructions precede shared context, limiting prefix reuse across call sites. The claim remains unchanged, evidence accumulates, and queries and individual search results are replaced between iterations.

prompt followed by the instructions for decomposition (A), search (B), assessment (C), or verdict generation (D). Their cache retention and eviction policies manage these identified prefixes without distinguishing the sharing patterns and lifetimes of individual components in the dynamic suffix. The claim flows unchanged to all four call sites and remains useful throughout the session. The evidence collection is shared by assessment and verdict calls and preserves accumulated content as it grows. Queries and individual search results are replaced between iterations. These differences remain hidden when the dynamic suffix is treated as opaque.

First, this structure enables prompt layouts that expose more prefix reuse. In Figure 1, each call site's fixed instructions can be reused across its invocations, but differing instructions before the shared claim prevent prefix reuse across sites. Moving the claim immediately after the common system prompt exposes a shared prefix across all four sites. Placing accumulated evidence before temporary inputs similarly exposes its unchanged token prefix for reuse in later calls. Once these prefixes are exposed, the serving engine's existing prefix caching mechanism can reuse them. **Second, this structure enables retention decisions based on future uses.** The claim will be consumed by later calls, whereas cached states for an iteration's temporary tails may never be reused. Figure 2 illustrates retaining shared prefixes while releasing disposable tails. Although a search result's content is appended to the evidence collection, its cached states in the original prompt position need not remain reusable. Component relationships must therefore be interpreted together with prompt order: layout determines which values form reusable prefixes, and lifecycle information helps determine how long those prefixes are worth retaining.

Our key insight is that the sharing patterns and lifetimes of program values provide actionable information for exposing additional prefix reuse and retaining cached states according to their future uses. More importantly, we observe that this sharing and lifetime information

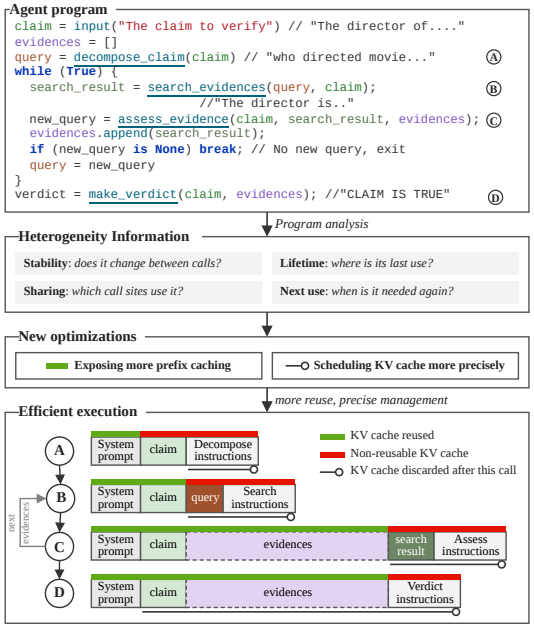


Figure 2. Program analysis recovers sharing, updates, and lifetime information for prompt components. Combined with runtime observations, this information guides coordinated prompt layouts and cache retention. Shared context moves forward, while discard markers identify cached tails that can be released after a call.

is already encoded in the agent program itself, making it recoverable through compiler analysis. Frameworks such as byLLM, NOOA, and DSPy (Furgale et al., 2026; Khattab et al., 2024; **?**) generate prompts from typed LLM function declarations and argument values, establishing a correspondence between prompt components and program variables. In Figure 2, dataflow connects the unchanged claim to all four call sites, the append operation exposes growing evidence, and assignments inside the loop identify replaced queries. Control flow identifies possible future consumers. Recovering these relationships makes it possible to coordinate layouts across calls and assess the future usefulness of cached prefixes, even before those future prompts arrive.

Turning this insight into efficient serving requires addressing three challenges. **First, program structure alone does not determine actual reuse.** Static analysis reveals possible sharing, updates, and future consumers, but execution paths, component sizes, and inter-call delays vary at runtime. **Second, exposing prefix reuse requires coordination across call sites.** Independently ordering each call's components can place the same value behind different preceding contexts, preventing its cached states from being shared. **Third, potential reuse must be weighed against current memory demand.** Retaining or prefetching a prefix for a future call consumes memory that other requests could use, while premature eviction forces recomputation.

We present MORTIS, a co-design of the compiler and serving runtime that addresses these challenges through a common description of prompt components and their uses. To connect program structure with actual execution, the compiler produces component metadata and a workflow graph, which the runtime supplements with observations to estimate future reuse and its timing. Using this information, *adaptive prompt re-layout* coordinates component ordering across related calls to expose longer shared prefixes. *Proactive KV-cache scheduling* then uses component lifetimes and predicted reuse to prioritize retention and eviction. It bounds retention with a fixed memory budget and prefetches only into available space, reclaiming disposable cached tails when needed. Together, these mechanisms connect the analysis of program values to both the construction of reusable prefixes and the allocation of memory to preserve them.

We make the following contributions: **(i)** We identify intra-prompt heterogeneity and the opportunities it creates to expose additional prefix reuse and distinguish the retention needs of prompt components. **(ii)** We develop compiler analysis to recover component sharing and lifetimes from agent programs, and combine this information with runtime observations to coordinate prompt layout and KV-cache management. **(iii)** We implement MORTIS and evaluate it on four agent workloads. It achieves geometric-mean speedups of 1.45× in average job completion time (JCT) and 1.49× in throughput over SGLang, with maxima of 1.99× and 1.97×, respectively.

2 BACKGROUND AND MOTIVATION

Prefix caching and KV-cache tiering Serving engines such as SGLang (Zheng et al., 2024) and vLLM (Kwon et al., 2023) reuse the longest matching prefix, computing only the remaining tokens. Tiered caching systems such as CachedAttention (Gao et al., 2024), Mooncake (Qin et al., 2025), and SGLang HiCache (SGLang Contributors, 2026) extend the KV-cache beyond device memory with host memory and optionally disk storage to improve capacity. However, off-device KV states must be transferred back to GPU memory before use.

KV-cache management for LLM agents Agent serving systems use recurring patterns in agentic workloads to manage the KV-cache. KVFlow (Pan et al., 2025) derives each agent's steps-to-execution from the workflow to guide eviction and prefetching of each agent's prompt prefix; Helium (Wadlom et al., 2026) analyzes the workflow to order LLM calls and prewarm static prefixes; Learned Prefix Caching (Yang et al., 2025), PBKV (Zheng et al., 2026) and CacheScout (Zhang et al., 2026) predict upcoming agent invocations with trained models to decide which caches to retain. KVTether (Fu et al., 2026) captures runtime lifecycle semantics from agent harness signals to preferentially preserve live-but-idle KV-cache. They operate the KV-cache on the identified prefix. Some systems, such as Pythia (Yu et al., 2026), prefill the predicted next invocations to proactively prepare KV-cache that does not exist yet.

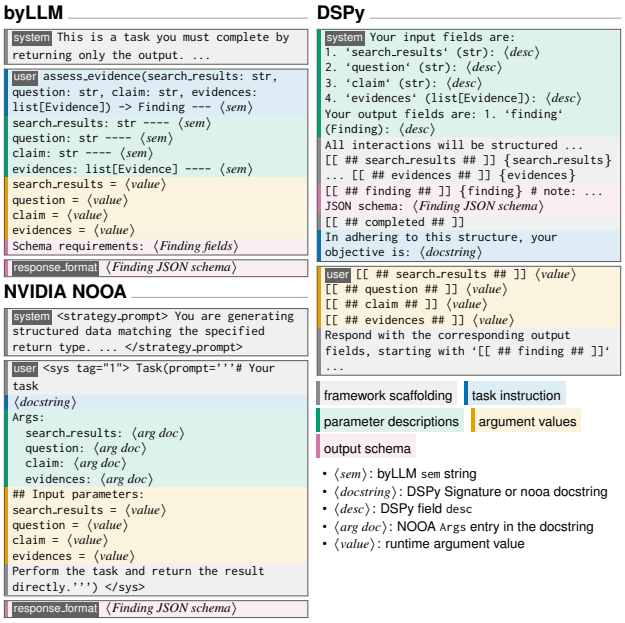


Figure 3. Prompts lowered by three frameworks for the same call share five kinds of components; only their order differs.

Frameworks with LLM calls as typed functions A class of agent frameworks exposes LLM invocations through typed programming interfaces, such as byLLM's meaning-typed programming (MTP) abstraction (Dantanarayana et al., 2025), DSPy's Signatures and Modules (Khattab et al., 2024), and NOOA's typed Python methods (Furgale et al., 2026). The developer declares a function or signature describing the desired behavior, inputs, and return type, and invokes it like an ordinary call within the program's control flow. At each invocation, the framework runtime lowers the declaration and its argument values into final prompts.

Although these agent frameworks differ in surface syntax, they lower LLM invocations in the same way: the intermediate representation is lowered with the runtime argument values to produce the prompt. As shown in Figure 3, these frameworks emit the same five kinds of components: framework scaffolding, task instruction, parameter descriptions, argument values, and output schema. The only difference is their relative positions.

Prompt-layout Opportunity: By default, component order follows framework templates and argument declarations. These layouts can be adjusted to expose more prefix reuse. For example, DSPy's static adapter (DSPy Contributors, 2026) lets users tweak the lowering process to put static prompt components at the front. NOOA (Furgale et al., 2026) lets developers label arguments as static prefix,

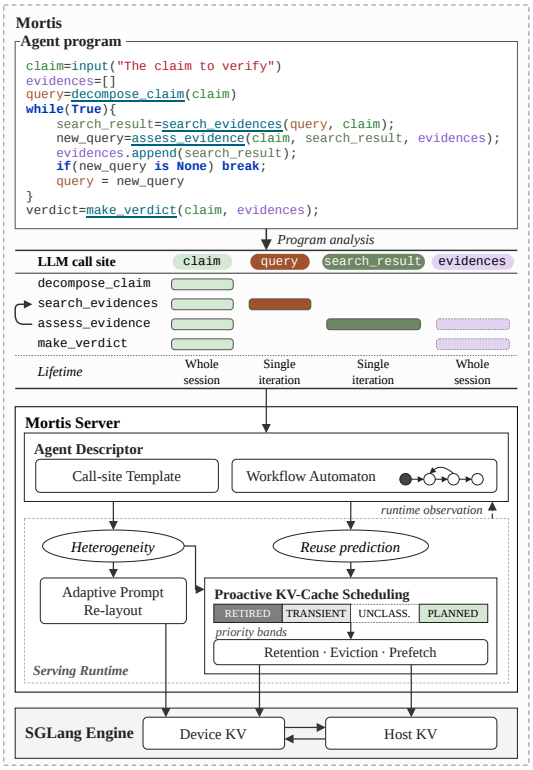


Figure 4. Architecture overview of MORTIS. Program analysis and runtime observations guide prompt re-layout and KV-cache management.

append-only history or dynamic suffix so that the framework can order them for better cache reuse.

Intra-prompt heterogeneity in programs Prompts contain components with different sharing patterns and lifetimes. In frameworks with typed LLM calls, components generated from arguments correspond to program variables. Program analysis can recover which values are shared, extended, or replaced, and identify possible future uses.

Cache-management Opportunity: Prior serving policies fail to fully exploit the reuse in agent workloads because they ignore the component-level structure of prompts. Manual annotations mitigating this gap like NOOA's labels restate properties that already exist in the program, and may become stale once the code is updated. Instead, reasoning about the program itself can precisely guide better prefix reuse and KV-cache retention decisions by discovering sharing and lifetime information.

From Deficiencies to Design Ignoring intra-prompt heterogeneity costs a serving system in two ways. First, shared tokens lose prefix caching: once a differing token appears ahead of them, their KV-cache can no longer be matched. Second, KV-cache retention loses precision: the system sees only a matched prefix and cannot manage entries by their individual reuse. Both costs are substantial. Among the agents we evaluated in §4, 51–92% of shared tokens miss the prefix cache because a differing token precedes them, and about 20% of served prompt tokens are used only once within a session. The layout freedom and the analyzability of intra-prompt heterogeneity address these deficiencies directly, upon which MORTIS is built.

3 DESIGN

In this section, we present the two-tier compiler-serving co-design of MORTIS. As shown in Figure 4, the *agent descriptor* is generated by combining analysis of the agent program with observation of runtime behaviors. The server side then exploits the derived descriptor to schedule the KV-cache efficiently, using *adaptive prompt re-layout* (§3.3) and *proactive KV-cache scheduling* (§3.4).

3.1 Agent Descriptor

The *agent descriptor* contains the metadata for scheduling. Static analysis provides its skeleton, and the server refines the weights with runtime observations. An agent descriptor contains two parts: (i) **Call-site Template:** It provides a heuristic for each component's lifetime and sharing pattern. (ii) **Workflow Automaton:** It contains the program-level structure of an agent, including workflow dependencies, execution time estimation and transition probabilities.

3.1.1 Call-site Template

Prompts can be decomposed into heterogeneous components. We classify these components into the following two categories: (i) **Invariant Specifications:** the part of prompt that is fixed once the program is compiled. They are rendered from the compiler's own intermediate representation and stay constant. (ii) **Dynamic Bindings:** the argument values of LLM functions, which are determined at runtime.

We define the call-site template as $\mathcal{T}_c = \langle \mathcal{I}_c,\ \mathcal{B}_c,\ \mathcal{H}_c \rangle$, where $\mathcal{I}_c$ denotes the invariant specifications, $\mathcal{B}_c$ the dynamic bindings, and $\mathcal{H}_c$ the heterogeneity annotation over those bindings.

Heterogeneity annotation For each dynamic binding, $\mathcal{H}_c$ records how its value is shared and how long that value stays valid, which we represent as $\mathcal{E}$ and $\mathcal{S}$ respectively. $\mathcal{S}$ is the set of program regions that overwrite a binding, and $\mathcal{E}$ classifies a binding into five classes (Table 1) by reusability.

3.1.2 Workflow Automaton

We abstract an agent's workflow as a time-annotated probabilistic finite-state automaton. We formulate it as:

$$\mathcal{A} = (Q, P, q_0, F, D^{\text{gap}}),$$

Table 1. Evolution classes of a dynamic binding, from the most to the least reusable.

Class	Value relationship
CONST	Fixed by the code and identical at every call.
COPY	Same as the value another call-site carries.
RESP	Comes from the reply produced by another call-site.
EXTEND	Append-only context; the previous value remains a prefix of the new one.
VOLATILE	Freshly produced at each call, or its origin lies outside the analysis boundary.

where: (i) Q is the set of states. Each state $q \in Q$ is labeled by an LLM call-site; (ii) P denotes the transition probabilities between states; (iii) $q_0 \in Q$ is the entrance call-site; (iv) F specifies the termination probability of the agent at each state; (v) $D^{\text{gap}}(q, q')$ is the inter-call gap distribution between the completion of call-site q and the invocation of its successor q'..

3.2 Constructing the Agent Descriptor

Call-site graph construction The compiler identifies LLM call-sites from the typed AST. It then builds the interprocedural control-flow graph (CFG) of the agent and projects it onto these call-sites. An edge (q, q') is added whenever q' is directly reachable in one hop from q, while non-LLM function call-sites are contracted. If a call target cannot be resolved statically, the compiler conservatively adds edges to all candidate targets. This projected graph forms the unweighted structure of $\mathcal{A}$. Finally, the compiler applies Bourdoncle's weak topological ordering (Bourdoncle, 1993) to recover the loop nesting structure from the cycles in $\mathcal{A}$.

Separating $\mathcal{I}_c$ and $\mathcal{B}_c$ $\mathcal{I}_c$ and $\mathcal{B}_c$ are obtained by running the lowering process that generates actual prompts. The compiler runs the framework's lowering pass ahead of time using symbolic placeholders for the arguments to produce the static portion of the runtime prompt ($\mathcal{I}_c$) while leaving one labeled placeholder per parameter to represent $\mathcal{B}_c$.

Recovering evolution classes ($\mathcal{E}$) For each binding $b \in \mathcal{B}_c$, the compiler computes the reaching definitions of its underlying variable across the CFG (including back edges, with local definitions shadowing earlier ones). Each reaching definition is classified by its right-hand side: (i) literals and module-level constants are classified as **CONST**; (ii) variables passed unmodified from an earlier call-site c' are classified as **COPY**(c'); (iii) values assigned from the return value of another call-site c' (traced via def–use chains) are classified as **RESP**(c'); (iv) prefix-preserving updates, like string/list concatenation ($v \leftarrow v + \Delta$), are classified as **EXTEND**; (v) all other definitions are defaulted to **VOLATILE**, such as external input.

Algorithm 1 Layout decision

Input: Descriptor $(\mathcal{A}, \{\mathcal{T}_q\}_{q\in Q})$, switch *pin*, lengths ℓ
Output: Layouts $\{\mathcal{L}_q\}_{q\in Q}$

$B_{\text{sh}} \leftarrow \{b : |\{q \in Q : b \in \mathcal{B}_q\}| > 1\}$ ▷ shared bindings
$G \leftarrow (B_{\text{sh}}, \emptyset)$ ▷ precedence graph
for all $(q, q') \in \textsc{Edges}(\mathcal{A})$ **do**
  $P_1, P_2, P_3 \leftarrow \emptyset$
    ▷ Groups: unchanged, append, rewritten/one-sided
  for all $b \in B_{\text{sh}} \cap (\mathcal{B}_q \cup \mathcal{B}_{q'})$ **do**
    $P_{\tau(b)} \leftarrow P_{\tau(b)} \cup \{b\}$ ▷ $\tau(b)$: group assignment of b
  $E(G) \leftarrow E(G) \cup \{(b, b') \in P_i \times P_k : i < k\}$
$\mathcal{C} \leftarrow \textsc{Tarjan}(G)$ ▷ SCCs of G, sets of bindings
$\hat{E} \leftarrow \{(C, C') \in \mathcal{C}^2 : C \neq C',\ (C \times C') \cap E(G) \neq \emptyset\}$
$\hat{G} \leftarrow (\mathcal{C}, \hat{E})$ ▷ condensation graph of G
$j(b) \leftarrow (-\ell(b), \mathit{depth}(b)),\ \forall b \in B_{\text{sh}}$
$J(C) \leftarrow \min_{b\in C} j(b),\ \forall C \in V(\hat{G})$
$L \leftarrow [\,]$
while $V(\hat{G}) \neq \emptyset$ **do** ▷ topological sort
  $R \leftarrow \{C \in V(\hat{G}) : \mathit{indeg}(C) = 0\}$
  $C^* \leftarrow \arg\min_{C\in R} J(C)$
  $L \leftarrow L \parallel \textsc{Sort}(C^*, j)$; $\hat{G} \leftarrow \hat{G} - C^*$
for all $q \in Q$ **do**
  $\sigma_q \leftarrow [\, b \in L : b \in \mathcal{B}_q \,] \parallel \textsc{Sort}(\mathcal{B}_q \setminus B_{\text{sh}}, K)$
  $s_q \leftarrow \textsc{Spec}(\mathcal{I}_q)$ ▷ instruction specification of q
  if $\neg \mathit{pin} \wedge (\mathcal{B}_q \cap B_{\text{sh}} \neq \emptyset)$ **then**
    $\mathcal{L}_q \leftarrow \sigma_q \parallel s_q$ ▷ spec last
  else
    $\mathcal{L}_q \leftarrow s_q \parallel \sigma_q$ ▷ spec first
return $\{\mathcal{L}_q\}_{q\in Q}$

If different CFG paths yield conflicting evolution classes for b, the compiler falls back to **VOLATILE**. To let the runtime resolve this conservativeness once the actual transition (q, q') is known, the compiler attaches transition-specific metadata to each edge (q, q'). Specifically, it records which and how the variables are modified along (q, q'), and attaches the exact evolution class for each binding at q'. For example, a binding can be reduced to **CONST** rather than **VOLATILE** if the exact edge assigns it a literal.

Recovering lifetime scopes ($\mathcal{S}$) The compiler records $\mathcal{S}(b)$ as the set of procedures (or loop bodies) that overwrite the binding b's underlying variable. When the variable is assigned once and remains valid for the entire session, $\mathcal{S}(b) = \emptyset$. Whenever execution enters a procedure in $\mathcal{S}(b)$, the variable is overwritten and its previously cached KV bytes become dead.

3.3 Adaptive Prompt Re-layout

3.3.1 Movable Components

First, the dynamic bindings $\mathcal{B}_c$ can be permuted freely. The compiler lowers each argument into a self-contained component labeled by its parameter name, and the framework places these components in declaration order only because that is the order of the argument list; re-layout may choose any order of the bindings.

Second, the instruction specification in $\mathcal{I}_c$ is movable. $\mathcal{I}_c$ consists of the system prompt, the response schema, the tool schema, and the instruction specification (such as task descriptions and parameter descriptions). The system prompt stays universally constant and should be placed at the very beginning, and the schemas are handled by the constrained decoding engine. The instruction specification can be put just after the system prompt or at the end of the prompt.

3.3.2 *Layout Decision*

MORTIS decides the layout $\mathcal{L}_q$ for each call-site q by ranking the reusability of bindings (Algorithm 1). It places shared bindings ahead of private bindings and prioritizes stable bindings within both categories to maximize prefix cache efficiency.

Sorting shared bindings (lines 1–17) Shared bindings are those that are carried by multiple call-sites. For every transition (q, q'), we identify how each shared binding evolves. We categorize their behaviors into three sharing pattern groups, ordered as follows: **(i) unchanged:** Both call-sites carry it and its value does not change between the two calls. **(ii) append:** Both call-sites carry it and the program only appends to its value. **(iii) rewritten/one-sided:** The program rewrites it between the two calls, or only one call-site carries it.

MORTIS builds the precedence graph of all shared bindings based on the groups they belong to on each transition edge, condenses it into a DAG using Tarjan's algorithm, and finally performs a topological sort on the DAG. We key each SCC by the smallest **(*-length*, *depth*)** among its bindings, where: **(i)** *depth*: the number of transitions from the entrance to the first call-site that carries b; **(ii)** *length*: the token length of the value of b, observed by the server. The SCC with the smallest key goes first when multiple SCCs have zero in-degree. Each SCC is then expanded into the list of its bindings sorted also by **(*-length*, *depth*)**.

Sorting private bindings (line 19) The bindings carried by a single call-site are sorted in ascending lexicographic order of:

$$K(b) = \big(\mathit{scoped},\ \rho,\ \mathit{depth},\ -\mathit{length}\big),$$

where: **(i)** *scoped*: the lifetime of b; 0 if it is session-wide ($\mathcal{S} = \emptyset$) and 1 otherwise; **(ii)** ρ: the stability rank for $\mathcal{E}$ in Table 1, from 1 for CONST to 5 for VOLATILE.

Placing the specification (lines 20–24) Placing the *instruction specification* at the end gives a longer common prefix. MORTIS places the specification at the end for each call-site that carries a shared binding. A switch keeps the specification at the front when the developer requires it.

3.4 Proactive KV-cache Scheduling

3.4.1 *Reusable Head and Volatile Tail*

A served prompt naturally divides into two segments: a reusable head and a volatile tail. The reusable head consists of two parts: a *static prefix* and a *session-shared extension*: (i) **Static Prefix** is the prefix that does not change, including $\mathcal{I}_c$ and the CONST bindings that re-layout has placed behind $\mathcal{I}_c$. (ii) **Session-shared Extension** consists of the bindings after the static prefix whose values recur unchanged in a later call of the same session.

3.4.2 *Next-Call Prediction*

MORTIS predicts three aspects of the next call: (i) The probability of the arrival. (ii) When the call will arrive. (iii) Which prefix of its prompt can be speculatively constructed.

Arrival probability Each call-site q can be located by a chain of loops enclosing it; for each call-site's invocation, we mark it with a tuple $\zeta = (i_1, i_2, \ldots, i_m)$, where $i_k, k \in [1, m]$ is the current iteration index of the k-th enclosing loop, from outermost to innermost.

MORTIS estimates the probability of the next call by a loop-tagged table. We define the table as $\mathcal{P}(q' \mid q, \zeta)$ and $\mathcal{F}(q, \zeta)$, where $\mathcal{P}$ is the probability that call-site q in context ζ is followed by call-site q'; $\mathcal{F}$ stands for the probability that the session ends at call-site q under context ζ. With an empty ζ (no loop context), $\mathcal{P}$ and $\mathcal{F}$ reduce to the base transition and termination probabilities P and F of the automaton $\mathcal{A}$.

The table is approximated and updated by frequency counts observed at runtime. Let q and ζ denote the call-site and loop context of the preceding call in the same session. When a call at q' arrives, the server increments the count of outcome q' in row (q, ζ); when the session closes at q, it increments the termination count in row (q, ζ). Each observation is also counted in every row (q, ζ_k) with $\zeta_k = (i_{m-k+1}, \ldots, i_m)$, $k = 0, \ldots, m$.

After a call at q lands, we compute the probability of each outcome o by Witten–Bell interpolation (Witten & Bell, 1991) from the empty context ζ_0 to the full context $\zeta_m = \zeta$. The outcome o ranges over $\mathrm{succ}(q)$, the static successors of q plus a virtual outcome end for session termination:

$$\hat{P}_k(o) = \lambda_k \frac{c_k(o)}{n_k} + (1 - \lambda_k)\hat{P}_{k-1}(o), \quad k = 0, \ldots, m,$$

$$\hat{P}_{-1}(o) = \frac{1}{|\mathrm{succ}(q)|}, \qquad \lambda_k = \begin{cases} \dfrac{n_k}{n_k + d_k} & n_k > 0, \\ 0 & n_k = 0, \end{cases}$$

$c_k(o)$ is how many times outcome o followed a call at q in context ζ_k, $n_k = \sum_o c_k(o)$ is the total number of observations in that row, and d_k is the number of distinct

outcomes with $c_k(o) > 0$. Then $\mathcal{P}(q' \mid q, \zeta) = \hat{P}_m(q')$ and $\mathcal{F}(q, \zeta) = \hat{P}_m(\text{end})$. A context with no observation is skipped. The weight λ_k grows with n_k, so a frequently observed context dominates the prediction, while a rarely observed one defers to its shorter contexts.

Arrival time The time of arrival is conservatively approximated based on recorded statistics. The workflow automaton records $D^{\text{gap}}(q, q')$ per transition: each time a call at q' arrives, the server appends the interval between the reply of the preceding call at q and this arrival. The interval is the agent's own time spent in its code between two calls. We estimate the time gap to the next call as:

$$\hat{g} = \max\{t_{50} - \tfrac{1}{2}(t_{90} - t_{50}), 0\}$$

where t_{50} and t_{90} are the 50th and 90th percentiles of the gap samples on the transition (q, q'); The second term ensures that a transition with a more dispersed gap distribution receives a tighter estimate.

Prefix reconstruction MORTIS constructs the prefix of call-site q' for matching KV-cache entries to schedule. Since $\mathcal{H}_c$ records the relationships among arguments, MORTIS builds the prefix in layout $\mathcal{L}_{q'}$ using the argument values observed in the current session. The construction stops at the first argument with an unknown value.

3.4.3 KV-cache Eviction Using Lifecycle Bands

Bands We categorize KV-cache entries into four bands, from first evicted to last: (1) **RETIRED:** the volatile tails of prompts belonging to completed sessions. (2) **TRANSIENT:** the volatile tail of a served prompt of a live session. (3) **UNCLASSIFIED:** entries that analysis cannot categorize. They keep the engine's default treatment. (4) **PLANNED:** prefixes that a predicted call will reuse.

Band transitions Bands are assigned at three moments of a session's lifecycle. (1) **When a served prompt enters the cache**, MORTIS splits it at its *reusable head* (§3.4.1). The volatile tail and the reply generated after it become *TRANSIENT*; the head stays *UNCLASSIFIED* until the next prediction refresh finds a successor call that starts with it, which tries to promote it to *PLANNED*. (2) **When a later call arrives**, the transition it takes may rewrite bindings that earlier heads carried. Each affected head is cut back to the prefix that survives the rewrite, and the bytes past it are demoted to *TRANSIENT*. (3) **When the session ends**, the volatile tails of every prompt it served drop to *RETIRED*. Their heads stay *UNCLASSIFIED* and age out by recency, since other sessions of the same program may share them.

Band-ordered eviction Eviction empties the lowest band first and falls back to recency within the three lower bands. The radix tree's eviction heap is keyed by band, then by recency. The host tier keeps LRU eviction, except that *RETIRED* entries go first. Additionally, KV-cache entries marked as *TRANSIENT* are not admitted into the host tier as they will not be reused.

3.4.4 KV-cache Retention and Prefetch

A future reuse prediction is summarized as the following triple:

$$\textit{Prediction} = (\pi_{q'},\ \mathcal{P}(q' \mid q, \zeta),\ t_{\text{reply}} + \hat{g})$$

where $\pi_{q'}$ is the prefix reconstructed for q' and t_{reply} is the time the current call replied. Predictions are refreshed at every call, and are dropped when the predicted calls arrive or their predicted arrival time passes.

MORTIS acts on this triple in two ways: *Retention* keeps $\pi_{q'}$ from leaving the device, and *prefetch* restores the part of $\pi_{q'}$ that has already been demoted to the host.

Retention MORTIS ranks the predicted reusable KV-cache entries by the benefit of keeping $\pi_{q'}$ resident at every refresh:

$$\mathcal{D}(q') = \frac{\mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}}{\max(t_{\text{reply}} + \hat{g} - t_{\text{now}},\ \tau_{\min})}$$

where $|\pi_{q'}|_{\text{cached}}$ is the number of cached tokens of $\pi_{q'}$, and $\tau_{\min}$ is the floor that prevents an imminent call from receiving an unbounded $\mathcal{D}$. MORTIS admits predictions into *PLANNED* in descending order of $\mathcal{D}$ until they cover a fixed fraction of the device tier. Within *PLANNED*, each radix-cache node accumulates the $\mathcal{D}$ of every planned prefix that covers it, so a shared head outranks a private one.

Prefetch When part of $\pi_{q'}$ has been demoted to the host, the planner prefetches it back to the device ahead of the estimated arrival time. The prefetch release deadline is:

$$t_{\text{release}} = t_{\text{reply}} + \hat{g} - \frac{|\pi_{q'}|_{\text{host}}}{B_{\text{h2d}}} - \frac{|\pi_{q'}|_{\text{uncached}}}{B_{\text{prefill}}}$$

where B_{h2d} and B_{prefill} are the measured host-to-device copy rate and prefill rate, $|\pi_{q'}|_{\text{host}}$ is the number of tokens of $\pi_{q'}$ cached only on the host, and $|\pi_{q'}|_{\text{uncached}}$ is the number of uncached tokens that require prefill.

Among released predictions, the one with earliest deadline goes first, with ties broken by the expected saving $\mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}$. A prefetch is released when the engine is idle or its deadline t_{release} approaches. It takes only free device memory, after reclaiming *RETIRED* and *TRANSIENT* entries. When free space is insufficient, prefetching pauses but keeps refreshing the recency of these cache entries so that they do not age out.

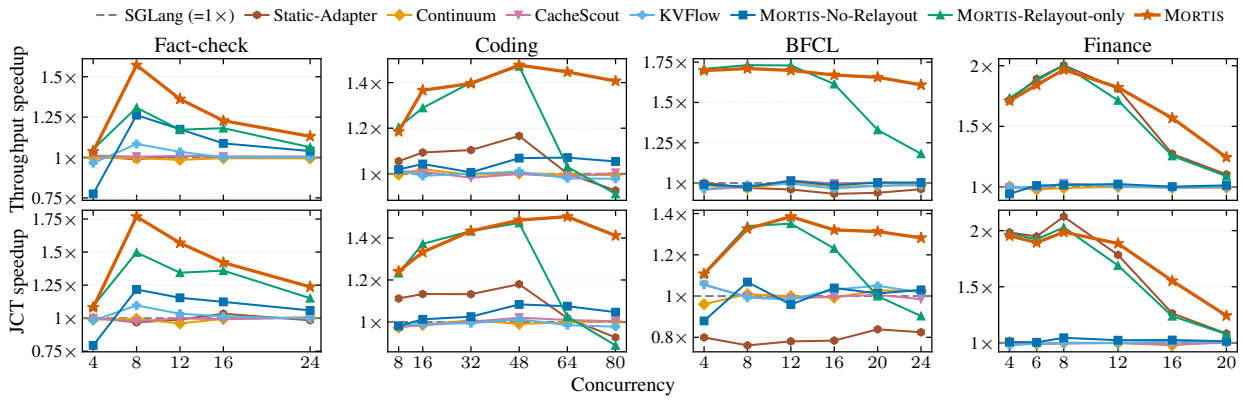


Figure 5. Throughput speedup (top) and average JCT speedup (bottom) over SGLang on each agent workload under varying concurrency.

4 EVALUATION

4.1 Experimental Setup

Testbed and model We implement MORTIS on top of SGLang v0.5.10.post1 and Jac (Mars et al., 2023) compiler, and conduct evaluations on one A40 GPU using Qwen3-32B-AWQ. We allocate 15 GB of GPU memory and 25 GB of CPU memory to the device and host KV-cache pools.

Baselines We compare MORTIS against five baselines, reimplemented on our SGLang fork: **(i) SGLang** (Zheng et al., 2024), the vanilla SGLang engine with HiCache and default LRU eviction; **(ii) CacheScout** (Zhang et al., 2026), which learns call transitions online to prioritize and prefetch agent prefixes; **(iii) Continuum** (Li et al., 2025), which pins agent KV-cache across tool gaps using adaptive TTLs; **(iv) KVFlow** (Pan et al., 2025), which schedules prefix eviction and prefetching via an Agent Step Graph; and **(v) Static-Adapter**, which mirrors DSPy's adapter (DSPy Contributors, 2026) by hoisting invariant components to the front under a fixed, program-agnostic layout.

We also evaluate the following ablation variants of MORTIS: (i) **MORTIS-Relayout-only** enables adaptive prompt re-layout but replaces proactive KV-cache scheduling with LRU eviction; (ii) **MORTIS-No-Relayout** enables proactive KV-cache scheduling but disables adaptive prompt re-layout; and (iii) **MORTIS-No-Prefetch** disables only KV-cache prefetching. Additionally, we run baseline serving systems on top of MORTIS's re-layout to isolate scheduling policy under the same prompt layout.

Workloads We evaluate MORTIS on four representative agent workloads, ported to Jac byLLM: a HoVer-style (Jiang et al., 2020) fact-check agent, a HumanEval (Chen et al., 2021) coding agent, a BFCL (Patil et al., 2025) tool-use agent, and a FinanceBench (Islam et al., 2023) financial-analysis agent. We further evaluate a mixed workload comprising all four agent types to assess MORTIS's end-to-end performance under concurrent, heterogeneous demand. We report the average job completion time (JCT) and throughput (prompt tokens served per minute).

4.2 End-to-End Performance of MORTIS

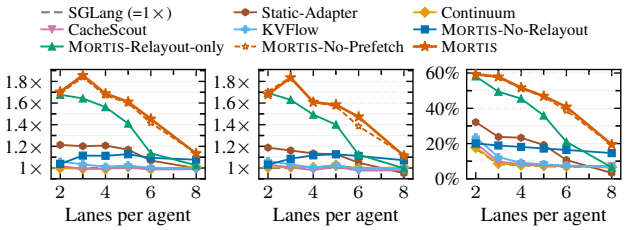
Performance on different workload types As shown in Figure 5, MORTIS achieves a 1.08–1.99× JCT speedup and increases throughput by 1.04–1.97× compared with SGLang, with geometric-mean gains of 1.45× in JCT and 1.49× in throughput. Across these agents, serving systems such as KVFlow show no significant improvement, because they ignore intra-prompt heterogeneity.

Under low concurrency, adaptive prompt re-layout accounts for nearly all of the improvement, because memory is sufficient. As load increases, however, the KV-cache becomes contested. Without knowing what to keep and what to give up, other serving systems fail to schedule it efficiently.

For the financial-analysis agent, KV-cache scheduling adds little, because this agent is a simple pipeline that digests large blobs of documents. Under low pressure, adaptive prompt re-layout accounts for nearly all of the improvement by exposing larger prefix reuse. MORTIS's proactive KV-cache scheduling only takes effect under the highest load, because it can proactively evict stale KV-cache entries.

Performance on mixed workload We evaluate MORTIS under varying pressure on the mixed workload and report metrics over each system's steady-state window. We vary the number of lanes (concurrent workflows) per agent, keeping it equal across all four agents.

As shown in Figure 6, MORTIS achieves up to 1.86× throughput speedup and 1.83× JCT speedup over SGLang at 3 lanes per agent. Under low pressure, adaptive prompt re-layout alone accounts for nearly all of the gain, since device memory is still large enough to retain the reusable KV-cache. As pressure grows, the cache becomes contested and the gain from re-layout alone shrinks, while proactive KV-cache scheduling retains a clear advantage by protecting the reusable heads that LRU would evict. Under the heaviest



(a) Throughput speedup (b) JCT speedup (c) Cache hit rate

Figure 6. Throughput and JCT speedup over SGLang and cache hit rate under varying lanes per agent on the mixed workload. JCT speedup is the geometric mean of the four agents' speedups.

Table 2. Task accuracy under three prompt layouts that appear in our evaluation.

Prompt layout	Fact-check	Coding	Financial-analysis	BFCL
Original byLLM	16.3%	55.3%	34.9%	14.1%
MORTIS	21.9%	54.8%	37.2%	18.7%
Static-Adapter	16.3%	52.2%	37.8%	16.0%

pressure, configurations converge toward the baseline, as the engine is fully saturated and no free memory is left for cache management to exploit. Across the six lane counts, MORTIS achieves geometric-mean speedups of 1.56× in throughput and 1.53× in JCT.

In the mixed workload, agents have heterogeneous demands for KV-cache memory, which reduces the chance that they contend for memory at the same time. By exploiting intra-prompt heterogeneity, MORTIS can promptly yield stale KV-cache to other requests that need memory, which further amplifies the benefit of MORTIS.

Task quality The performance gains of MORTIS do not come at the cost of task quality. As shown in Table 2, across three different layouts produced by the original byLLM, Static-Adapter, and MORTIS, reordering components does not harm task quality. Moreover, the three layouts differ by at most 3% in LLM calls per task and 6% in output tokens. MORTIS's speedup is not the result of less work either.

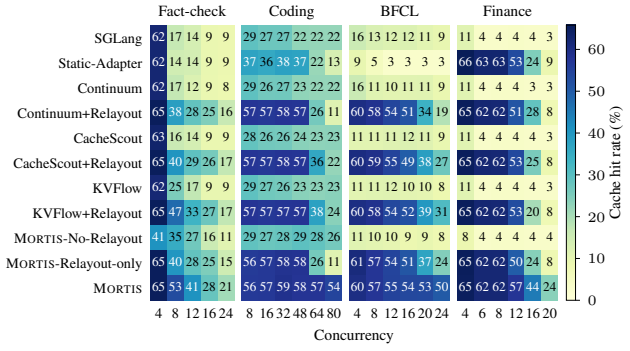


Figure 7. Cache hit rate (%) of prompt tokens under varying concurrency

Table 3. Geometric-mean speedup over SGLang across all concurrency levels of the four agent workloads, with *Adaptive Prompt Re-layout* and *Proactive KV-cache Scheduling* on or off.

(a) JCT speedup

	Scheduling	
Re-layout	Off	On
Off	1.00×	1.03×
On	1.30×	1.45×

(b) Throughput speedup

	Scheduling	
Re-layout	Off	On
Off	1.00×	1.02×
On	1.36×	1.49×

4.3 Cache Efficiency of MORTIS

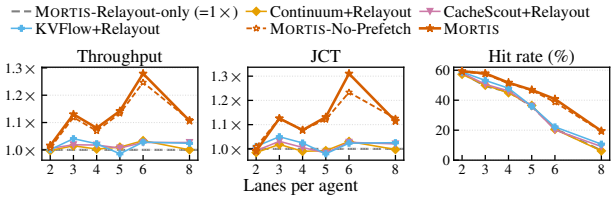
Figure 7 shows the cache hit rate of prompt tokens at the same concurrency levels as Figure 5. On Coding and BFCL, adaptive prompt re-layout alone brings the hit rate at low concurrency to that of MORTIS: it places the stable and shared bindings at the front, where they form a reusable head that later calls hit. As concurrency grows, the hit rate of MORTIS-Relayout-only drops quickly, because LRU eviction does not distinguish reusable heads from volatile tails. With proactive KV-cache scheduling, MORTIS keeps a higher hit rate than MORTIS-Relayout-only as concurrency grows, and on Coding its hit rate stays nearly flat. On the Fact-check agent, MORTIS's hit rate exceeds that of MORTIS-Relayout-only from the lowest concurrency. Continuum and CacheScout stay close to SGLang at every concurrency. MORTIS-No-Relayout stays below MORTIS, since under the original prompt order the reusable head is short. The mixed workload shows a similar pattern (Figure 6).

4.4 Ablation Study

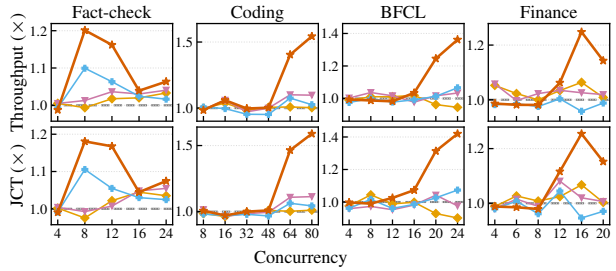
As the geometric-mean speedups of JCT and throughput over SGLang on the four workloads in Table 3 show, *adaptive prompt re-layout* and *proactive KV-cache scheduling* are superadditive: when both are enabled, the speedup is higher than the product of the two individual speedups. The same effect appears on the mixed workload (Figure 6a), where adding proactive KV-cache scheduling on top of adaptive prompt re-layout substantially improves performance.

The gain of MORTIS's KV-cache scheduling builds on re-layout, because re-laying out the prompt order exposes a reusable prefix and a suffix that can be abandoned. MORTIS's KV-cache scheduling substantially improves performance on top of re-layout, but scheduling alone (MORTIS-No-Relayout) barely helps (Table 3). The same trend appears in Figures 6 and 5.

Prefetching contributes little on this testbed. As shown in Figure 6a, disabling it (MORTIS-No-Prefetch) lowers throughput by at most 2.6% across the six lane counts. The gain of proactive KV-cache scheduling mainly comes from retention, which keeps the reusable heads resident.



(a) Throughput, JCT speedup and hit rate on the mixed workload



(b) Throughput and JCT speedup on each agent workload

Figure 8. Baseline systems served with MORTIS's re-laid prompts. Speedups are measured against MORTIS-Relayout-only.

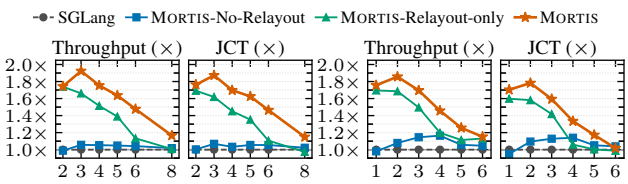
4.5 Comparison under Identical Layouts

Serving the baseline systems with re-laid prompts identical to MORTIS's, we evaluate their speedup against MORTIS-Relayout-only. As shown in Figure 8a, these systems show marginal improvement over MORTIS-Relayout-only, but are still beaten by MORTIS. We further evaluate on the four agent workloads separately. Figure 8b shows the same trend as in Figure 8a. The largest gains are from CacheScout on Coding and KVFlow on Fact-check, both up to 1.11× in JCT, but MORTIS still outperforms all of them.

This deficiency in baseline serving systems arises because they manage the KV-cache only at the granularity of a prefix and its suffix, which ignores intra-prompt heterogeneity. They cannot identify precisely which KV-cache entries will be reused, leading to inefficient KV-cache scheduling.

4.6 Sensitivity Study

We run the mixed workload on four A40 GPUs with TP=4 and the same KV-cache pool size configurations. TP=4 increases the prefill speed by 1.74× (from 1292.4 to 2251.8 tokens/s), which lowers the cost of a cache miss. As shown in Figure 9, MORTIS maintains its advantage even under cheaper cache misses. It achieves geometric-mean speedups of 1.60× in throughput and 1.58× in JCT (up to 1.92× and 1.87×). Additionally, we evaluate on a smaller GPU with a smaller model. On one RTX 3090 with Qwen3-14B-AWQ, with 5.2 GB of GPU memory and 10 GB of CPU memory for the KV-cache pools, MORTIS keeps the same pattern of improvement (Figure 9): re-layout exposes additional prefix cache reuse, and KV-cache scheduling takes effect once the cache becomes contested and thrashes.



(a) A40 TP=4 (b) RTX 3090

Figure 9. Throughput and JCT speedup over SGLang on the mixed workload under varying lanes per agent.

5 Related Work

Application-level information for serving Parrot (Lin et al., 2024) represents intermediate results as semantic variables and uses their producer–consumer relationships for scheduling and prefix-aware placement. Teola (Tan et al., 2025) exposes operators finer than a call, removes unnecessary dependencies, computes available prefixes while upstream operators produce the remaining inputs, and forwards partial outputs downstream.

Prompt construction for prefix reuse DSPy's adapters and NOOA fix a component order by convention or by developer labels (DSPy Contributors, 2026; Furgale et al., 2026), and the SGLang frontend moves constant components forward at compile time (Zheng et al., 2024). For relational analytics, Liu et al. (2025) order rows and the fields within them jointly, grouping rows that share field values and using functional dependencies to expose further sharing. Reuse can also go beyond matching prefixes: Prompt Cache (Gim et al., 2024) precomputes prompt modules at prescribed positions, and CacheBlend (Yao et al., 2025) recomputes the tokens whose attention crosses chunk boundaries when combining cached chunks.

KV-cache scheduling across execution gaps Serving systems also steer the KV-cache according to tool-call gaps in ReAct agents. InferCept (Abhyankar et al., 2024) weighs the cost of keeping, swapping, and recomputing KV-cache per interruption to make scheduling decisions; Continuum (Li et al., 2025) holds the idle KV-cache for a time-to-live derived from observed inter-invocation delays; TokenCake (Bian et al., 2026) estimates the tool call's duration, offloads idle KV to the host, and restores it before the expected return.

6 Conclusion

We present MORTIS, a co-design of the compiler and serving runtime that drives more efficient KV-cache scheduling by exploiting intra-prompt heterogeneity. It uses runtime-assisted static analysis and steers the KV-cache through prompt re-layout and proactive scheduling. Our evaluation shows that MORTIS achieves up to 1.99× JCT speedup and 1.97× higher throughput over SGLang. MORTIS highlights the importance of the structure inside prompts for serving.

REFERENCES

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 81–95. PMLR, 2024. URL https://proceedings.mlr.press/v235/abhyankar24a.html.

Bian, Z., Wu, F., Li, Z., Ma, T., and Zhuo, Y. TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications, 2026. URL https://arxiv.org/abs/2510.18586v5. Version 5. Accepted at EuroSys 2027.

Bourdoncle, F. Efficient chaotic iteration strategies with widenings. In *Formal Methods in Programming and Their Applications*, volume 735 of *Lecture Notes in Computer Science*, pp. 128–141. Springer, 1993. doi: 10.1007/BFb0039704.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dantanarayana, J. L., Kang, Y., Sivasothynathan, K., Clarke, C., Li, B., Kashmira, S., Flautner, K., Tang, L., and Mars, J. Mtp: A meaning-typed language abstraction for ai-integrated programming. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):1176–1204, 2025. doi: 10.1145/3763092.

DSPy Contributors. ChatAdapter: DSPy api reference. Online documentation, 2026. URL https://dspy.ai/api/adapters/ChatAdapter/. Accessed September 14, 2026.

Fu, K., Zhou, Y., Chang, C., Yu, Y., Wang, L., Yang, G., Shi, J., Chen, Q., and Wang, W. Capture the lifecycle: KV cache management in ReAct agents with KVTether, 2026. URL https://arxiv.org/abs/2609.39819.

Furgale, P., Klingler, S., Nolan, J., Staats, M., Di Lorenzo, G., Martinez Abad, E., Schüller, C., Dinu, R., Devoto, A., Berard, P., Kaplun, G., Sarafian, E., Roveri, R., Derczynski, L., and Silveira Cabral, R. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents, 2026. URL https://arxiv.org/abs/2607.20709.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 111–126, Santa Clara, CA, 2024. USENIX Association. URL https://www.usenix.org/conference/atc24/presentation/gao-bin-cost.

Gim, I., Chen, G., seob Lee, S., Sarda, N., Khandelwal, A., and Zhong, L. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 325–338, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., and Vidgen, B. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

Jiang, Y., Bordia, S., Zhong, Z., Dognin, C., Singh, M., and Bansal, M. HoVer: A dataset for many-hop fact extraction and claim verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3441–3460, 2020.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180.

Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2025.

Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium*

on Operating Systems Design and Implementation (OSDI 24), pp. 929–945. USENIX Association, 2024.

Liu, S., Biswal, A., Kamsetty, A., Cheng, A., Schroeder, L. G., Patel, L., Cao, S., Mo, X., Stoica, I., Gonzalez, J. E., and Zaharia, M. Optimizing LLM queries in relational data analytics workloads. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/b5dc49f44db2fadc5c4d717c57f4a424-Abstract-Conference.html.

Mars, J., Kang, Y., Daynauth, R., Li, B., Mahendra, A., Flautner, K., and Tang, L. The jaseci programming paradigm and runtime stack: Building scale-out production applications easy and fast. *arXiv preprint arXiv:2305.09864*, 2023.

Pan, Z., Patel, A., Hu, Z., Shen, Y., Guan, Y., Li, W.-L., Qin, L., Wang, Y., and Ding, Y. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows, 2025. URL https://arxiv.org/abs/2507.07400.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025.

Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, Santa Clara, CA, 2025. USENIX Association. URL https://www.usenix.org/conference/fast25/presentation/qin.

SGLang Contributors. HiCache system design and optimization. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/hicache_design. Accessed September 22, 2026.

Tan, X., Jiang, Y., Yang, Y., and Xu, H. Teola: Towards end-to-end optimization of LLM-based applications. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 1302–1316. Association for Computing Machinery, 2025. doi: 10.1145/3676641.3716278. URL https://arxiv.org/abs/2407.00326.

Wadlom, N., Shen, J., and Lu, Y. Efficient LLM serving for agentic workflows: A data systems perspective, 2026. URL https://arxiv.org/abs/2603.16104.

Witten, I. H. and Bell, T. C. The zero-frequency problem: Estimating the probabilities of novel events in adaptive text compression. *IEEE Transactions on Information Theory*, 37(4):1085–1094, 1991.

Yang, D., Li, A., Li, K., and Lloyd, W. Learned prefix caching for efficient LLM inference. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1526. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/414f642a1ea9350006669774cba9bcd4-Abstract-Conference.html.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3689031.3696098. URL https://www.microsoft.com/en-us/research/uploads/prod/2024/09/eurosys25-final999.pdf.

Yu, S., Shu, J., Ni, Y., Qian, K., Li, X., Wang, Y., Zhang, J., Xu, Z., Yang, S., Zhu, L., Zhai, E., Lu, Q., Xing, J., Lu, Y., Jin, X., Liu, X., and Xu, H. Pythia: Exploiting workflow predictability for efficient agent-native LLM serving, 2026. URL https://arxiv.org/abs/2604.25899.

Yuan, Y., Nayak, A., Kundu, S., and Talati, N. Agentic AI workload characteristics. In *2026 IEEE International Symposium on Workload Characterization (IISWC)*, 2026. URL https://arxiv.org/abs/2605.26297.

Zhang, R., Kim, C., Feng, S., Du, K., Liu, Y., Zhong, Y., Ching, C.-W., Jiang, J., and Hu, L. Learning agent execution for kv-cache management in agentic serving, 2026.

Zheng, H., Fu, F., Wu, J., Yuan, B., Zhang, Y., Wang, H., Zhu, Y., Yan, X., and Jiang, J. Efficient serving for dynamic agent workflows with prediction-based KV-cache management, 2026. URL https://arxiv.org/abs/2605.06472.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

AI USAGE DISCLOSURE

In this paper, we use Claude Code for prototyping and methodology investigation and organization. During system development, we use AI to help with algorithm design and implementation. In the paper-writing process, we

equip Claude Code with grammar-checking skills to clean up glitches in writing.